

Si tu veux réaliser un **parser Cub3D robuste**, il faut considérer que **tout ce qui n'est pas explicitement autorisé est une erreur**.

---

## 1. Vérification des arguments

### Nombre d'arguments

`./cub3D map.cub`

Erreur si :

- aucun argument
- plusieurs arguments

---

### Extension

Le fichier doit se terminer exactement par

`.cub`

Erreurs :

map  
map.txt  
map.cub.txt  
.cub

---

### Le fichier existe

Erreur si

`open() == -1`

---

### Le fichier est vide

Erreur si :

0 octet

ou uniquement

`\n`

`\n`

---

## 2. Lecture du fichier

Pendant la lecture :

- erreur de malloc

- erreur de get\_next\_line

doivent être gérées.

---

### 3. Les six paramètres obligatoires

On doit trouver exactement :

NO  
SO  
WE  
EA  
F  
C

---

#### Paramètre manquant

Exemple :

NO wall.xpm  
SO wall.xpm  
WE wall.xpm  
EA wall.xpm  
F 220,100,0

Il manque

C

Erreur.

---

#### Paramètre en double

NO a.xpm  
NO b.xpm

Erreur.

Même chose pour

F

ou

C

---

#### Identifiant inconnu

AB test

Erreur.

---

#### 4. Vérification des textures

Pour chaque texture :

NO ./textures/wall.xpm

---

##### Le chemin est vide

NO

Erreur.

---

##### Le fichier n'existe pas

NO ./wall.xpm

si le fichier est absent.

---

##### Le fichier n'est pas lisible

open()

échoue.

---

##### Mauvaise extension

wall.png  
wall.jpg  
wall.bmp

Erreur.

Les textures doivent être en **.xpm**.

---

#### 5. Vérification des couleurs

Une couleur est

R,G,B

---

##### Il manque une composante

220,100

Erreur.

---

### Il y en a quatre

220,100,50,10

Erreur.

---

### Valeur non numérique

220,a,30

Erreur.

---

### Valeur hors limites

300,10,10

Erreur.

-1,20,30

Erreur.

Chaque valeur doit être comprise entre 0 et 255.

---

### Virgules incorrectes

220,,30

Erreur.

220,

Erreur.

,220,30

Erreur.

---

### Espaces

Autoriser :

220, 30,40

220 ,30 , 40

si ton parser le souhaite.

---

## 6. Début de la map

La map doit être le dernier élément du fichier.

Après le début de la map :

```
11111
10001
11111
```

NO wall.xpm

Erreur.

---

## 7. Lignes vides

Les lignes vides avant la map :

OK.

Exemple

NO wall

SO wall

F 0,0,0

Valide.

---

## Les lignes vides à l'intérieur de la map

```
111111
```

```
100001
```

Erreur.

---

## 8. Caractères autorisés dans la map

Seulement

```
0
1
```

N  
S  
E  
W  
(espace)

Tout autre caractère :

2  
A  
x  
#

Erreur.

---

## 9. Joueur

Il doit y avoir exactement :

N  
  
ou  
  
S  
  
ou  
  
E  
  
ou  
  
W

---

**Aucun joueur**

Erreur.

---

**Deux joueurs**

N....S

Erreur.

---

## 10. La map doit être fermée

C'est probablement la vérification la plus importante.

Le joueur ne doit jamais pouvoir sortir.

Exemple faux

11111  
10001  
10000  
11111

Le zéro touche le vide.

Erreur.

---

Autre exemple

111111  
10 001  
111111

Le zéro touche un espace.

Erreur.

---

Cette vérification est généralement réalisée avec un **Flood Fill**.

---

## 11. Espaces

Les espaces sont autorisés.

Mais un

0

ne doit jamais toucher

''

sinon le joueur pourrait sortir.

---

## 12. Bordures

Aucun

0

ni

N  
S  
E

W

ne doit être :

- sur la première ligne
  - sur la dernière ligne
  - sur la première colonne
  - sur la dernière colonne
- 

### 13. Longueur des lignes

Les lignes peuvent avoir des tailles différentes.

Exemple valide

```
111111
1001
1111111111
```

Il faut gérer cela correctement.

La solution classique est de créer une copie **rectangulaire** remplie d'espaces.

---

### 14. Erreurs mémoire

Toujours vérifier :

```
malloc
calloc
ft_strdup
ft_substr
```

---

### 15. MLX

Après le parsing :

`mlx_init()`

peut échouer.

`mlx_new_window()`

peut échouer.

`mlx_xpm_file_to_image()`



peut échouer.

Il faut alors libérer toute la mémoire avant de quitter.

---

## 16. Fermeture du programme

À chaque erreur :

- fermer les fichiers ouverts ;
  - détruire les images MLX déjà chargées ;
  - détruire la fenêtre si elle existe ;
  - détruire le display MLX ;
  - libérer toutes les allocations ;
  - quitter avec un message d'erreur.
- 

## 17. Vérifications bonus

Tu peux également vérifier :

- chemin de texture contenant uniquement des espaces ;
  - couleurs avec espaces en fin de ligne ;
  - caractères après les couleurs (`F 0,0,0 abc`) ;
  - caractères après le chemin (`NO wall.xpm abc`) ;
  - tabulations mélangées aux espaces ;
  - retour chariot Windows (`\r\n`) ;
  - fichier contenant uniquement des espaces.
- 

## Ma checklist personnelle (ordre recommandé)

1. ☒ Arguments
2. ☒ Extension `.cub`

3. ☒ Ouverture du fichier
4. ☒ Fichier non vide
5. ☒ Lecture complète
6. ☒ Présence des 6 paramètres
7. ☒ Aucun doublon
8. ☒ Chemins des textures
9. ☒ Couleurs RGB
10. ☒ Début de la map
11. ☒ Aucun caractère interdit
12. ☒ Un seul joueur
13. ☒ Aucune ligne vide dans la map
14. ☒ Rectangularisation de la map
15. ☒ Map fermée (Flood Fill)
16. ☒ Chargement des textures
17. ☒ Initialisation MLX
18. ☒ Gestion propre des erreurs et de la mémoire

Cette liste couvre pratiquement toutes les erreurs qu'un correcteur Cub3D est susceptible de tester, ainsi que les cas limites qui provoquent souvent des fuites mémoire ou des plantages pendant les évaluations.